ECE276B Project 1 Report: Pathfinding With Dynamic Programming

Stephen Huang
Department of Electrical and Computer Engineering
University of California, San Diego
La Jolla, U.S.A.
sth004@ucsd.edu

***Abstract*—We present a dynamic programming approach to solving the Door & Key navigation problem. The environment consists of an agent in a grid world that must reach a goal location while accounting for walls, orientation-dependent motion, key possession, and a locked or open door. We consider non-uniform action costs so that the objective is not only to reach the goal, but to reach it with minimum total cost. We formulate the problem as a deterministic Markov Decision Process (MDP), where each state encodes the agent position, orientation, whether the key has been collected, and whether the door is open. For known maps, we compute a separate optimal policy for each provided environment. For random maps, we augment the state with the possible key and goal configurations so that one policy can be used across all random environments. Our approach applies deterministic shortest path dynamic programming to compute the optimal cost-to-go and extract the corresponding control policy. This produces optimal action sequences for both known and random Door & Key environments.**

I. Introduction

Autonomous navigation is important in robotics because a robot must determine the optimal sequence of actions needed to move from its current state to a desired goal. These actions must also account for obstacles, cost, and other intermediate tasks. For example, a robot may need to retrieve an object or open a door before reaching its destination. Therefore, autonomous navigation can be viewed as the problem of finding a minimum-cost plan that satisfies both motion and task constraints.

In this project, we study this problem using the Door & Key environment. The agent moves in a discrete grid world containing walls, a key, a door, and a goal location. The agent can move forward, turn left, turn right, pick up the key, and unlock the door. Each action has a different cost, so the objective is not only to reach the goal, but to reach it with minimum total cost. If the door blocks the path to the goal, the agent must first navigate to the key, pick it up, unlock the door, and then continue to the goal.

To solve this problem, we first formulate the environment as a deterministic Markov Decision Process (MDP), where the state includes the agent position, orientation, key possession status, and door status. Since the transition model is deterministic and all action costs are positive, the MDP problem becomes a deterministic optimal control (DOC) problem, which has a deterministic shortest path (DSP) equivalence. Therefore, we can use DSP to solve the problem. For known maps, we compute a separate optimal policy for each provided environment. For the random maps, we augment the state with the key and goal configuration so that a single policy can be used across all random maps.

II. Problem Formulation

Consider an agent navigating in a discrete Door & Key grid-world environment. The objective is to reach a goal cell while minimizing the total accumulated cost. The environment contains obstacles, a goal location, and may require the agent to pick up a key before unlocking a door.

We formulate this problem as a deterministic Markov Decision Process (MDP)

$$\mathcal{M} = (\mathcal{X}, \mathcal{U}, x_0, f, T, \ell, q), \tag{1}$$

where $\mathcal{X}$ is the state space, $\mathcal{U}$ is the control space, x_0, is the deterministic starting state, f is the deterministic transition model, ℓ is the stage cost, and q is the terminal cost. Since our transitions are deterministic, our next state is uniquely determined by the current state and selected action:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t). \tag{2}$$

A. State Space

We define our state space $\mathcal{X}$ with enough information to ensure it is Markovian. In particular, we define each state as

$$\mathbf{x}_t = \begin{bmatrix} p_t & d_t & k_t & s_t \end{bmatrix}, \tag{3}$$

where $p_t = (x_t, y_t)$ is the agent grid position, d_t is the agent orientation, k_t is the key possession variable, and s_t is the door status variable. The orientation is of the finite set

$$d_t \in \{\mathrm{N}, \mathrm{E}, \mathrm{S}, \mathrm{W}\}, \tag{4}$$

and the key and door variables are binary:

$$k_t \in \{0, 1\}, \qquad s_t \in \{0, 1\}. \tag{5}$$

When $k_t = 1$, the agent has the key, and when $s_t = 1$, the door is open.

For random maps, the key can appear in one of three positions and the goal can appear in one of three positions. Let E_{rand} be the set of possible environment configurations. Then, we can define the augmented state as

$$\mathbf{x}_t = \begin{bmatrix} p_t & d_t & k_t & s_t & \epsilon_t \end{bmatrix}, \tag{6}$$

where $\epsilon_t \in E_{\text{rand}}$ denotes the environment configuration, including the active key and goal locations.

B. Control Space

We define our control space $\mathcal{U}$. At each timestep, the agent selects one action from the control space

$$\mathcal{U} := \{\text{MF}, \text{TL}, \text{TR}, \text{PK}, \text{UD}\}, \tag{7}$$

where MF denotes move forward, TL denotes turn left, TR denotes turn right, PK denotes pick up key, and UD denotes unlock door.

We define the non-uniform stage cost as

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \begin{cases} 3, & \mathbf{u}_t = \text{MF}, \\ 1, & \mathbf{u}_t = \text{TL}, \\ 1, & \mathbf{u}_t = \text{TR}, \\ 2, & \mathbf{u}_t = \text{PK}, \\ 5, & \mathbf{u}_t = \text{UD}. \end{cases} \tag{8}$$

These values are formulated based off of energy usage and complexity constraints on the agent.

C. Motion Model

We define our motion model $f(x, u)$. We encode the cardinal directions

$$\text{N} = 0, \quad \text{E} = 1, \quad \text{S} = 2, \quad \text{W} = 3,$$

Then, the turning control input only updates the agent's orientation d_t:

$$d_{t+1} = d_t - 1 \pmod 4, \qquad \mathbf{u}_t = \text{TL}, \tag{9}$$

$$d_{t+1} = d_t + 1 \pmod 4, \qquad \mathbf{u}_t = \text{TR}. \tag{10}$$

The move forward control input attempts to move the agent one cell in the direction it is facing:

$$p_{t+1} = p_t + \Delta(d_t), \tag{11}$$

where $\Delta(d_t)$ is the grid displacement associated with the current orientation. If the cell in front of the agent is a wall or a locked door, then the agent remains in place:

$$p_{t+1} = p_t. \tag{12}$$

The PK action changes the key possession state only if the agent is facing the key:

$$k_{t+1} = \begin{cases} 1, & \mathbf{u}_t = \text{PK and key in front of agent,}, \\ k_t, & \text{otherwise.} \end{cases} \tag{13}$$

Similarly, the unlock-door action changes the door status only if the agent has the key and is facing a locked door:

$$s_{t+1} = \begin{cases} 1, & \mathbf{u}_t = \text{UD},\ k_t = 1,\ \text{and door in front of agent,} \\ s_t, & \text{otherwise.} \end{cases} \tag{14}$$

Invalid actions are modeled as self-transitions. The state remains unchanged, but cost is still incurred.

For the random maps, the same transition model is used but the environment configuration variable ϵ_t is static:

$$\epsilon_{t+1} = \epsilon_t. \tag{15}$$

D. Initial State and Terminal Set

The initial state is determined by the environment. For the known case, the initial condition is

$$\mathbf{x}_0 = \begin{bmatrix} p_0 & d_0 & 0 & 0 \end{bmatrix}, \tag{16}$$

where the agent begins at position p_0, orientation d_0, without the key, and with the door closed.

For random maps, the initial state additionally includes the environment configuration:

$$\mathbf{x}_0 = \begin{bmatrix} p_0 & d_0 & 0 & s_0 & \epsilon_0 \end{bmatrix}, \tag{17}$$

where s_0 is the initial door status and ϵ_0 specifies the active key and goal positions.

The terminal set is the set of states in which the agent has reached the goal:

$$\mathcal{X}_G := \{\mathbf{x} \in \mathcal{X} \mid p = p_{\text{goal}}\}. \tag{18}$$

For random maps, the goal position is determined by the configuration ϵ, so the terminal set is

$$\mathcal{X}_G := \{\mathbf{x} \in \mathcal{X} \mid p = p_{\text{goal}}(\epsilon)\}. \tag{19}$$

We define terminal costs such that the end goal must be reached:

$$q(\mathbf{x}) = \begin{cases} 0, & \mathbf{x} \in \mathcal{X}_G, \\ +\infty, & \mathbf{x} \notin \mathcal{X}_G. \end{cases} \tag{20}$$

E. Deterministic Optimal Control Objective

Given the MDP $\mathcal{M}$, we want to find a policy

$$\pi : \mathcal{X} \to \mathcal{U} \tag{21}$$

that minimizes the total cost required to reach the goal. Let τ_G denote the first time that the trajectory reaches the terminal goal set:

$$\tau_G := \inf\{t \geq 0 \mid \mathbf{x}_t \in \mathcal{X}_G\}. \tag{22}$$

We define the value function of a policy π as the total accumulated cost from the initial state $\mathbf{x}$ until the goal is reached:

$$V^{\pi}_{\tau_G}(\mathbf{x}) = \sum_{t=0}^{\tau_G - 1} \ell(\mathbf{x}_t, \pi(\mathbf{x}_t)) + q(\mathbf{x}_G), \ \ \mathbf{x}_0 = \mathbf{x}, \ \ q(\mathbf{x}_G) = 0. \tag{23}$$

The associated objective for a given initial state $\mathbf{x}_0$ is

$$J(\pi) := V^{\pi}(\mathbf{x}_0). \tag{24}$$

Thus, we want to find an optimal control policy that minimizes the value function:

$$\pi^* = \arg\min_{\pi} J(\pi) = \arg\min_{\pi} V^{\pi}(\mathbf{x}_0). \tag{25}$$

Then, our control sequence will be a minimum cost sequence of actions that reach the terminal goal.

III. Technical Approach

We solve the Door & Key problem using deterministic shortest path dynamic programming. Since the transition model is deterministic and all action costs are positive, we consider this a deterministic optimal control (DOC) problem, which has an equivalent DSP problem. Therefore, we want to construct a graph $G = (V, E)$ and solve backwards through dynamic programming. We implement this with two planners. The first planner, `KnownPlanner`, computes a separate optimal policy for each known map. The second planner, `UnknownPlanner`, computes a single policy for all random maps.

A. Vertex Construction

We begin by constructing the vertex set V for all possible states. We intentionally exclude wall cells to reduce computation. Each vertex corresponds to one full state.

1) Known Map Vertices: For known maps, each environment has fixed wall, key, door, and goal locations. The known planner constructs vertices of the form

$$v := (x, y, d, k, s), \tag{26}$$

where (x, y) is the agent position, d is the orientation, k is the key possession status, and s is the door status. We loop through all non-wall grid cells and adds all combinations of orientation, key status, and door status:

$$d \in \{0, 1, 2, 3\}, \qquad k \in \{0, 1\}, \qquad s \in \{0, 1\}. \tag{27}$$

This gives the vertex set (which is equivalent to the state space).

$$V_{\text{known}} = \mathcal{X}_{\text{known}}. \tag{28}$$

The known planner then identifies all goal vertices by selecting every vertex whose position equals the goal position:

$$V_G = \{(x, y, d, k, s) \in V_{\text{known}} \mid (x, y) = p_{\text{goal}}\}. \tag{29}$$

All goal vertices are assigned value zero regardless of its other state variables.

2) Random Map Vertices: To compute a single control policy for all random environments, we augment the state with indices for the possible key and goal positions:

$$v := (x, y, d, k, s, i_k, i_g). \tag{30}$$

Here, $i_k \in \{0, 1, 2\}$ indexes one of the three possible key locations, and $i_g \in \{0, 1, 2\}$ indexes one of the three possible goal locations. We map indices to positions

$$p_{\text{key}}(0) = (2, 2), \quad p_{\text{key}}(1) = (2, 3), \quad p_{\text{key}}(2) = (1, 6), \tag{31}$$

and

$$p_{\text{goal}}(0) = (6, 1), \quad p_{\text{goal}}(1) = (7, 3), \quad p_{\text{goal}}(2) = (6, 6). \tag{32}$$

The random map contains two possible doors. We represent this using a two-bit state variable

$$s \in \{0, 1, 2, 3\}. \tag{33}$$

The first bit represents the first door and the second bit represents the second door.

Similar to the known map planner, we construct vertices by loop through all valid cases (excluding wall cells) states. The resulting vertex set is thus equivalent to the state space for random maps:

$$V_{\text{random}} = X_{\text{random}}. \tag{34}$$

The goal vertices are similarly defined by checking

$$V_G = \{(x, y, d, k, s, i_k, i_g) \in V_{\text{random}} \mid (x, y) = p_{\text{goal}}(i_g)\}, \tag{35}$$

and assigning a value of zero to each vertex in V_G.

B. Edge Construction

We construct edges for the graph via the motion model $f(\times x, u)$ and associated action cost $\ell(x, u)$. Consider the control space

$$\mathcal{U} = \{\text{MF}, \text{TL}, \text{TR}, \text{PK}, \text{UD}\} \tag{36}$$

For each action $u \in \mathcal{U}$ and vertex $v \in V$, we define a transition function that returns the next vertex and corresponding action cost:

$$(v', c) = \text{Transition}(v, u), \tag{37}$$

where

$$v' = f(v, u), \qquad c = \ell(v, u). \tag{38}$$

This creates an edge with an associated edge cost.

1) Orientation and Front Cell Computation: We encode the cardinal directions as integers;

$$0 = \text{North}, \quad 1 = \text{East}, \quad 2 = \text{South}, \quad 3 = \text{West}. \tag{39}$$

Each direction maps to a grid displacement:

$$\Delta(d) = \begin{cases} (0, -1), & d = 0, \\ (1, 0), & d = 1, \\ (0, 1), & d = 2, \\ (-1, 0), & d = 3. \end{cases} \tag{40}$$

For a state with position (x, y) and orientation d, the cell directly in front of the agent is

$$(x', y') = (x, y) + \Delta(d). \tag{41}$$

The front cell is used to determine whether the agent can move forward, pick up the key, or unlock a door.

2) Control: Turning: The turn left control input decrements the orientation by modulo four:

$$f((x, y, d, k, s), \text{TL}) = (x, y, (d - 1) \bmod 4, k, s), \tag{42}$$

while turning right increments the direction modulo four:

$$f((x, y, d, k, s), \text{TR}) = (x, y, (d + 1) \bmod 4, k, s). \tag{43}$$

The same update is used in the random map planner, except key and goal indices are preserved.

The associated action cost is

$$\ell(v, \{TL, TR\} = 1$$

.

3) Control: Move Forward: The move forward control input requires our planner to check the cell in front of the agent. If the front cell is a wall or a closed door, we create a self-transition edge

$$f(v, \mathrm{MF}) = v. \tag{44}$$

If the front cell is a closed door, the agent also remains in place. If the front cell is free or an open door, the agent moves into the cell:

$$f((x, y, d, k, s), \mathrm{MF}) = (x', y', d, k, s). \tag{45}$$

All cases will incur an action cost

$$\ell(v, MF) = 3.$$

This effectively gives invalid moves infinite cost.

4) Control: Pick Up Key: For picking up the key, the planner checks whether the key is directly in front of the agent. For known maps, we check equivalence with a global p_{key}. If the agent does not already have the key and the front cell contains the key, then the key status is updated:

$$f((x, y, d, k, s), \mathrm{PK}) = (x, y, d, 1, s). \tag{46}$$

Otherwise, the state remains unchanged:

$$f(v, \mathrm{PK}) = v. \tag{47}$$

For random maps, we first determine the key location through its index $p_{key}(i_k)$. Then, we compare it with the front cell

$$(x', y') = p_{\mathrm{key}}(i_k). \tag{48}$$

If this condition is true and $k = 0$, we have

$$(x, y, d, 1, s, i_k, i_g) = f((x, y, d, 0, s, i_k, i_g), PK).$$

otherwise, the state remains unchanged.

All cases will incur the action cost

$$\ell(v, \mathrm{PK}) = 2$$

5) Control: Unlock Door: For the unlock door action, our agent must satisfy the following conditions

$$(x', y') = p_{\mathrm{door}}, \quad k = 1, \quad s = 0$$

where p_{door} is the door position in the known map. Then, our next state becomes

$$(x, y, d, k, 1) = f((x, y, d, k, s), \mathrm{UD}). \tag{49}$$

If any of those conditions are not satisfied, the state remains unchanged.

In the random map case, there are two possible doors. The planner uses the door-status integer s as a bit mask. If the agent unlocks the first door, the planner updates

$$s' = s \mid 1, \tag{50}$$

where $\mid$ is the bitwise OR operation. If the agent unlocks the second door, the planner updates

$$s' = s \mid 2. \tag{51}$$

This allows the planner to track whether neither, either, or both doors have been opened.

The door unlock action incurs a cost

$$\ell(v, \mathrm{UD}) = 5.$$

C. Value and Policy Initialization

The planner stores the value function as a dictionary mapping each vertex to its current cost-to-go estimate:

$$\mathcal{V} : V \to \mathbb{R}_{\geq 0} \tag{52}$$

The value of every non-goal vertex is initialized to infinity:

$$V(v) = +\infty, \qquad v \notin V_G. \tag{53}$$

Every goal vertex is initialized to zero:

$$V(v) = 0, \qquad v \in V_G. \tag{54}$$

The policy is also stored as a dictionary:

$$\pi : V \to \mathcal{U}. \tag{55}$$

It is initialized as empty for all states and is filled in during value iteration by storing the action that minimizes the Bellman update.

D. Value Iteration for DSP

We use dynamic programming and iterate backwards from the goal vertices. For each non-goal vertex v, we compute

$$Q(v, u) = \ell(v, u) + V(f(v, u)). \tag{56}$$

The value of the vertex is then updated according to

$$V(v) \leftarrow \min_{u \in \mathcal{U}} \left[Q(v, u) \right]. \tag{57}$$

At the same time, the minimizing action is stored as the current policy:

$$\pi(v) \leftarrow \arg\min_{u \in \mathcal{U}} \left[\ell(v, u) + V(f(v, u)) \right]. \tag{58}$$

This update is repeated over all non-goal vertices until convergence. Since our DSP satisfies the following conditions:

- Every optimal path is simple,
- There are no negative-cost cycles,
- The graph has $|V|$ vertices,
- $V(v) = 0, \;\; v \in V_G$

it is guaranteed to converge in $|V| - 1$ iterations.

However, we also end the algorithm if it converges early. We define this as

$$\Delta_{\max} = \max_{v \in V \setminus V_G} |V_{\mathrm{new}}(v) - V_{\mathrm{old}}(v)| = 0 \tag{59}$$

Since the graph is finite and all action costs are positive, the value function converges to the minimum cost-to-go for each state that can reach a goal.

Algorithm 1 Deterministic Shortest Path Value Iteration

function VALUEITERATION($V, V_G, \mathcal{U}$)
  for each vertex $v \in V$ **do**
    $V(v) \leftarrow +\infty$
    $\pi(v) \leftarrow$ None
  end for
  for each goal vertex $v \in V_G$ **do**
    $V(v) \leftarrow 0$
  end for
  repeat
    $\Delta_{\max} \leftarrow 0$
    for each vertex $v \in V \setminus V_G$ **do**
      $V_{\text{old}} \leftarrow V(v)$
      $J_{\min} \leftarrow +\infty$
      $u_{\min} \leftarrow$ None
      for each action $u \in \mathcal{U}$ **do**
        $(v', c) \leftarrow$ TRANSITION(v, u)
        $J \leftarrow c + V(v')$
        if $J < J_{\min}$ **then**
          $J_{\min} \leftarrow J$
          $u_{\min} \leftarrow u$
        end if
      end for
      $V(v) \leftarrow J_{\min}$
      $\pi(v) \leftarrow u_{\min}$
      $\Delta_{\max} \leftarrow \max(\Delta_{\max}, |V(v) - V_{\text{old}}|)$
    end for
  until $\Delta_{\max} = 0$
  return V, π
end function

E. Action Sequence Extraction

After we have computed the optimal policy, we extract an action sequence by continuously applying the control policy to the initial state v_0. For known maps, we have

$$v_0 = (x_0, y_0, d_0, 0, 0), \tag{60}$$

where (x_0, y_0) is the initial agent position, d_0 is the initial orientation, the key possession variable is initialized to 0, and the door status is initialized as closed.

For the random maps, the initial state is

$$v_0 = (x_0, y_0, d_0, 0, s_0, i_k, i_g), \tag{61}$$

where s_0 is determined by the initial open or closed status of the two doors, i_k is determined by the observed key position, and i_g is determined by the observed goal position.

Starting from v_0, the planner repeatedly applies

$$u_t = \pi(v_t), \tag{62}$$

then transitions to

$$v_{t+1} = f(v_t, u_t), \tag{63}$$

until a goal vertex is reached:

$$v_t \in V_G. \tag{64}$$

The returned plan is the resulting action sequence

$$(u_0, u_1, \ldots, u_{T-1}). \tag{65}$$

IV. RESULTS

We evaluate the DSP planners on both the known and random maps.

A. Known Map Results

For known maps, each result has its own control policy.

1) 5x5 Normal Environment: Trajectory representation for 5x5 environment.

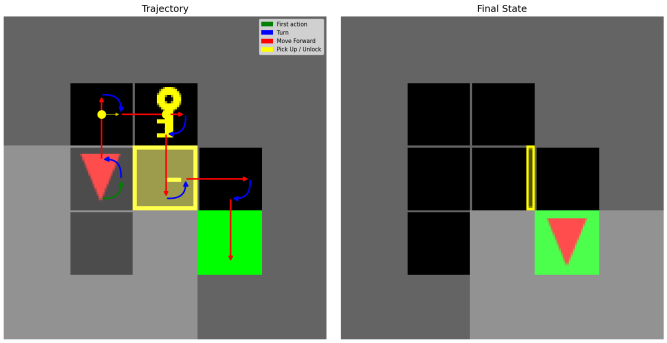


Fig. 1. Known map result for `doorkey-5x5-normal`.

2) 6x6 Environments: Trajectory representation for 6x6 environment.

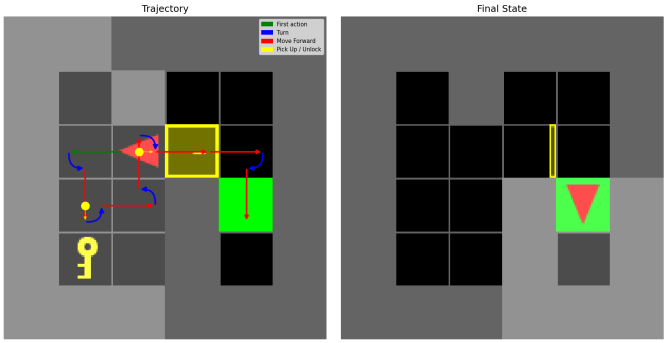


Fig. 2. Known map result for `doorkey-6x6-normal`.

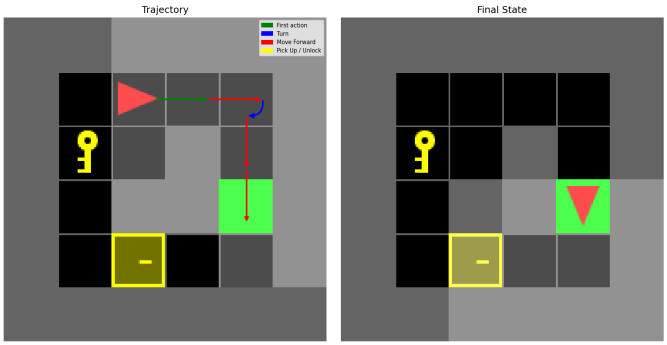


Fig. 3. Known map result for `doorkey-6x6-direct`.

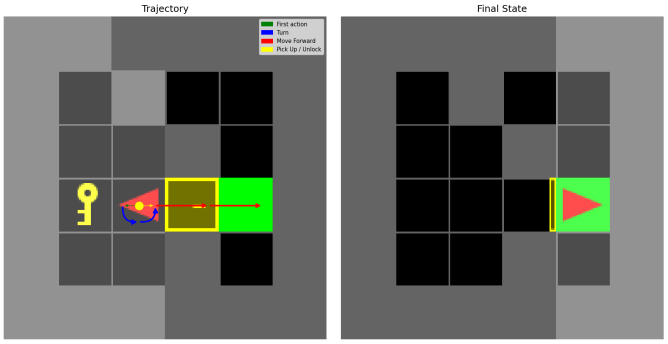


Fig. 4. Known map result for `doorkey-6x6-shortcut`.

3) 8x8 Environments: Trajectory representation for 8x8 environment.

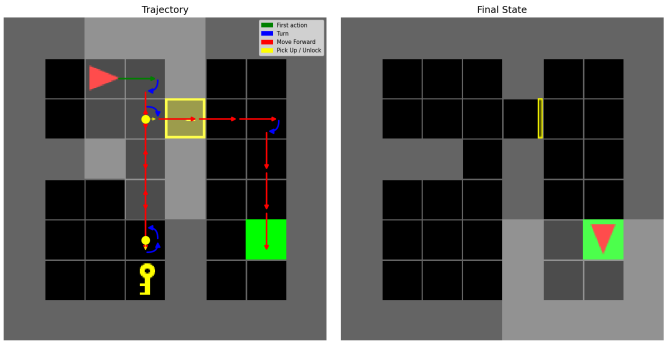


Fig. 5. Known map result for `doorkey-8x8-normal`.

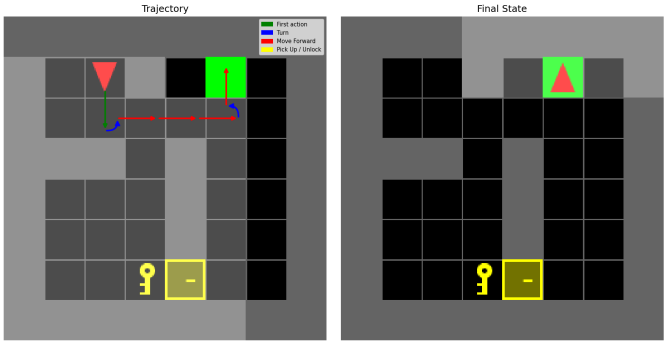


Fig. 6. Known map result for `doorkey-8x8-direct`.

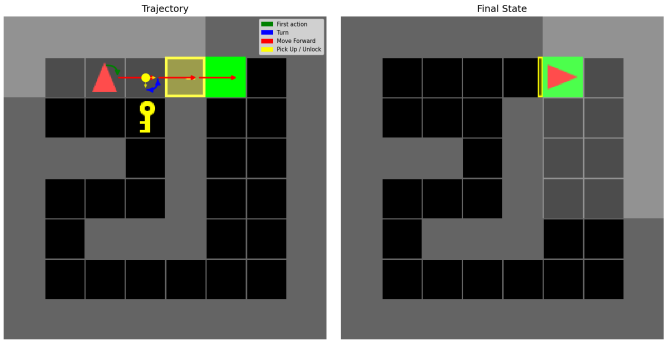
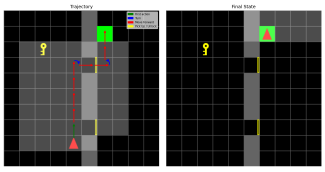


Fig. 7. Known map result for `doorkey-8x8-shortcut`.

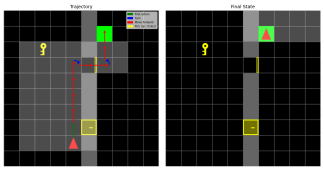
B. Random Map Results

For random maps, each figure is the result of one control policy.

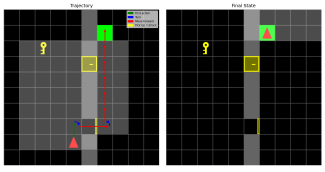
1) Random Environments 1–36: Trajectory representation for all random map environments.



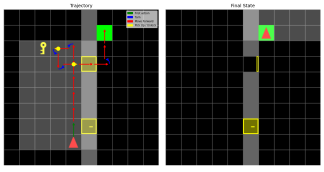
(a) `DoorKey-10x10-1`.



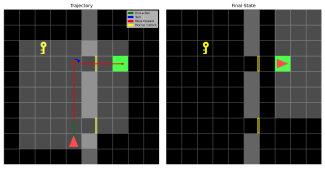
(b) `DoorKey-10x10-2`.



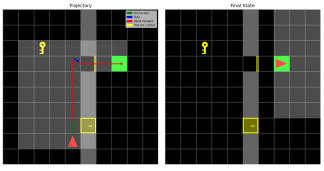
(c) `DoorKey-10x10-3`.



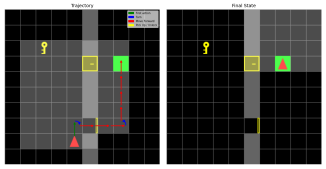
(d) `DoorKey-10x10-4`.



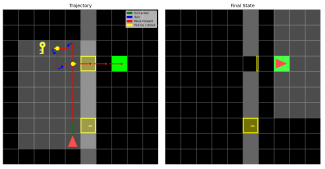
(e) `DoorKey-10x10-5`.



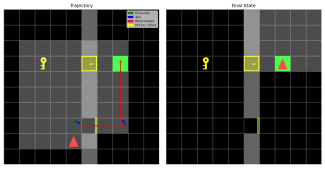
(f) `DoorKey-10x10-6`.



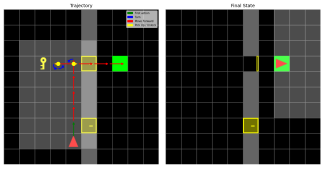
(g) DoorKey-10x10-7.



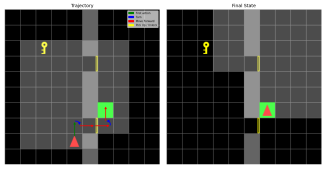
(h) DoorKey-10x10-8.



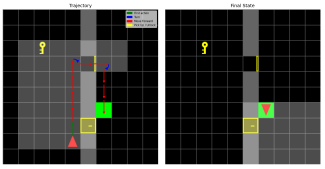
(s) DoorKey-10x10-19.



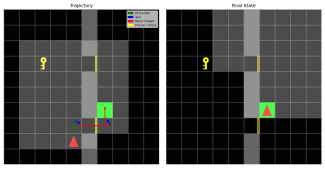
(t) DoorKey-10x10-20.



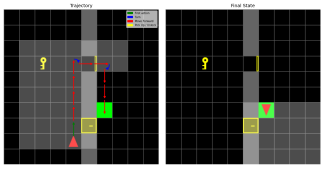
(i) DoorKey-10x10-9.



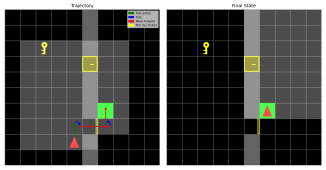
(j) DoorKey-10x10-10.



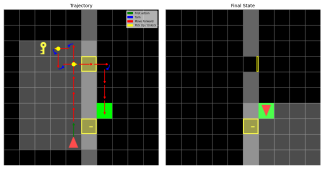
(u) DoorKey-10x10-21.



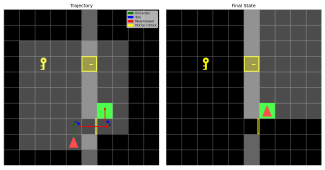
(v) DoorKey-10x10-22.



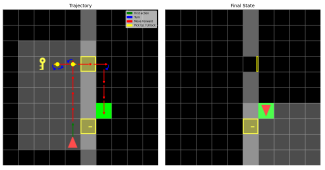
(k) DoorKey-10x10-11.



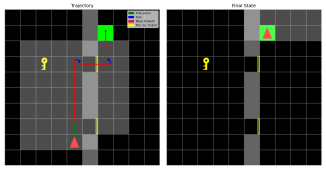
(l) DoorKey-10x10-12.



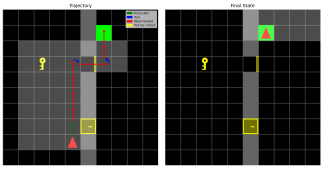
(w) DoorKey-10x10-23.



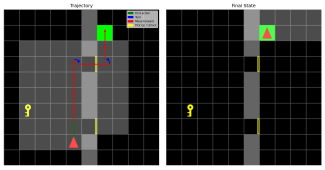
(x) DoorKey-10x10-24.



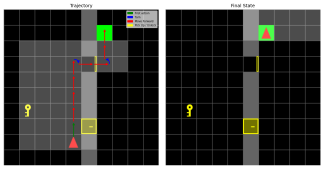
(m) DoorKey-10x10-13.



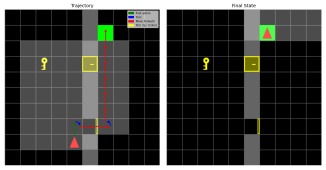
(n) DoorKey-10x10-14.



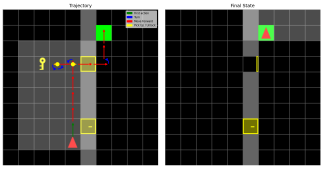
(y) DoorKey-10x10-25.



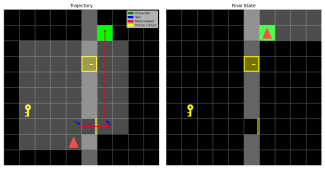
(z) DoorKey-10x10-26.



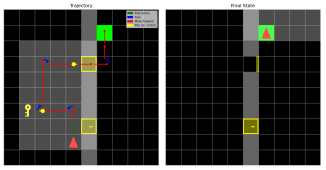
(o) DoorKey-10x10-15.



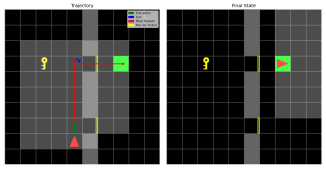
(p) DoorKey-10x10-16.



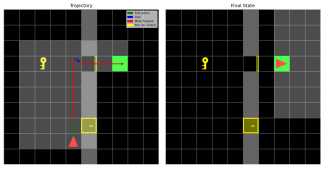
() DoorKey-10x10-27.



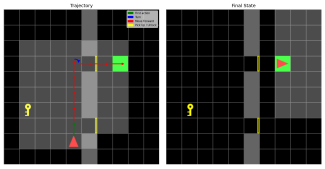
() DoorKey-10x10-28.



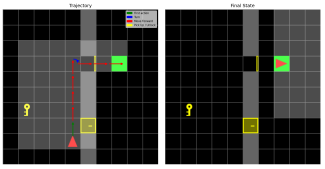
(q) DoorKey-10x10-17.



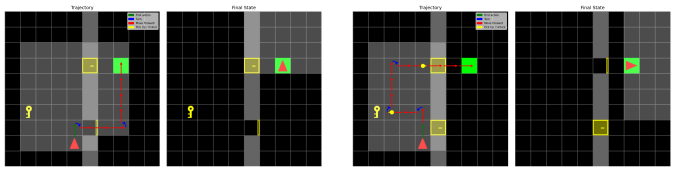
(r) DoorKey-10x10-18.



() DoorKey-10x10-29.

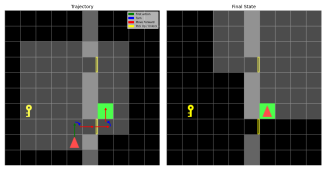


() DoorKey-10x10-30.

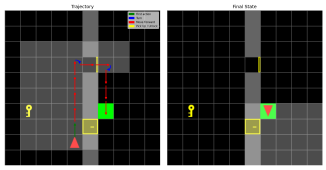


() DoorKey-10x10-31.

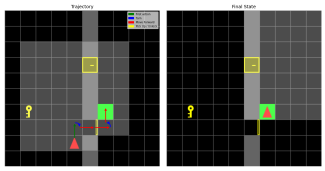
() DoorKey-10x10-32.



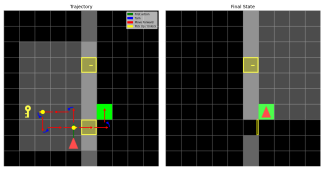
() DoorKey-10x10-33.



() DoorKey-10x10-34.



() DoorKey-10x10-35.



() DoorKey-10x10-36.

C. Discussion

1) Algorithm Performance: What worked?: The main optimizations used to speed up the algorithm were targeted towards reducing the number of vertices in the graph. If we were to consider the entire possible state space, a single cell can have 8 states: 4 orientation, 2 key states, and 2 door states. Furthermore, every cell can contain a key and every cell can be a door. If we consider an $n \times n$ grid, then the total number of vertices would be

$$8 \cdot n \cdot n \cdot n^2 = 8n^4$$

. Since our finite DSP algorithm is guaranteed to converge in $T = |V| - 1$ iterations, we have a big-O time complexity of

$$8n^4 \cdot (8n^4 - 1) = O(n^8).$$

Therefore, by reducing the number of cells we consider, we can significantly reduce computation. To do this, we choose to ignore cells that are marked as wall cells, as they are impossible to reach or start in for the agent. Then, if w is the number of wall cells, we only have to compute for $(n - w)$ cells instead of n cells. Furthermore, for known maps, only one cell can be a key and one cell can be a door. Therefore, by not enumerating every possible key & door position, we have

$$8 \cdot 1 \cdot 1 \cdot (n - w)^2 = 8(n - w)^2 \text{ total vertices}$$

and a time complexity of

$$8(n - w)^2 \cdot (8(n - w)^2 - 1) = O((n - w)^4),$$

significantly less than $O(n^8)$. Similarly, for random maps, we have 3 key positions and 3 door positions respectively. Therefore, we have

$$8 \cdot 3 \cdot 3 \cdot (n - w)^2 = 72(n - w)^2 \text{ total vertices}$$

and a time complexity of

$$72(n - w)^2 \cdot (72(n - w)^2 - 1) = O((n - w)^4),$$

which is also much less than $O(n^8)$.

Overall, the algorithm performed extremely well, formulating an optimal control policy in ≈ 0.1 seconds for most known maps and ≈ 4.5 seconds for random maps.

2) Algorithm Performance: What didn't work?: The main limitation that I encountered was that many non-useful actions (e.g. PK when no key in adjacent blocks, MF when wall is in front) were evaluated even if it had no meaningful result. That is, I was unable to remove the self-transition edges from my graph. This resulted in significant slow downs, especially when computing the random maps (≈ 2 second slow down). These self-transitions are difficult to remove because there are many edge cases that must be considered, and to evaluate every edge case for every state may instead increase the program time rather than reduce it.